Audit du code au regard du défi 3 « L'IA gardienne de la donnée »

Projet : qualif-comptage — branche `claude/brave-edison-tsewpy` **Date** : 14 novembre 2026
Périmètre : backend FastAPI (moteurs de qualité, de reconstruction, de diagnostic, services d'import, modèles, routers), frontend React (structure et gestion d'erreurs), CI.

1. Énoncé du défi (capture d'écran fournie)

> **Contexte** : la donnée énergétique d'un territoire est vivante et imparfaite : compteurs renommés, bâtiments rénovés, trous de mesure, valeurs estimées, données multifluides avec des fréquences de relève parfois saccadées. Toute décision — de crise ou de décarbonation — repose pourtant sur sa qualité. **Problématique** : une IA peut-elle veiller **en continu** sur la donnée énergétique d'un territoire — détecter les anomalies, **qualifier chaque valeur** et **tracer chaque hypothèse** ? **Exemples d'idées** : un agent de qualité des données : détection des anomalies, qualification (**mesuré / estimé / hypothèse**) et **score de confiance par série**, exploitable par les métiers.

2. Méthode et limites

- Lecture du code source (backend complet ; frontend : structure, gestion d'erreurs, exports).
- Exécution des 18 tests existants : **18 passés**.
- Deux scripts de sondage ont été exécutés sur une base SQLite temporaire :
- rejeu du parcours import → analyse → correction → validation, et cas de terrain ;
- benchmark de détection sur séries synthétiques étiquetées (2 ans, bruit 5 %, anomalies injectées).
- Le dépôt n'a pas été modifié par l'audit.

Limites - Les données du lien kDrive du défi n'ont pas été consultées : **tous les chiffres sont issus de données synthétiques**. - Le frontend n'a pas été validé à l'exécution : `npm ci` échoue (pas de `package-lock.json`), donc la vérification TypeScript n'est pas concluante. - L'extrapolation de performance (point 13) n'a pas été mesurée.

3. Conformité aux exigences du défi

#	Exigence	Verdict	Éléments
E1	Veiller **en continu**	**Absent**	Alimentation par dépôt manuel de CSV ; ni planificateur, ni connecteur, ni alerte. L'analyse est cloisonnée par lot (`import_batch_id`) : l'historique d'un compteur importé en plusieurs fois n'est jamais réuni.
E2	Détecter les anomalies	**Couvert**	Dix sous-types (pic, chute, négatif, palier, index en baisse, index plat en saison, zéro en saison, consommation haute/basse, horodatage irrégulier). Rappel de 100 % sur le benchmark. Réserve : règles fixes, pas d'apprentissage.

#	Exigence	Verdict	Éléments
E3	Qualifier chaque valeur : mesuré / estimé / hypothèse	**Partiel**	`ReadingQuality` = valide / incoherence / manquante / reconstruite. « Estimé » ≈ reconstruite ; « mesuré » et « hypothèse » n'existent pas.
E4	Tracer chaque hypothèse	**Partiel, insuffisant**	`raw_value` conservé ; historique au niveau du lot uniquement ; méthode par point calculée (`methods_used`) mais non enregistrée ; ni auteur, ni paramètres, ni version du moteur ; méthode écrite en dur « profil_type + interpolation ».
E5	Score de confiance par série	**Partiel**	Le score est `valides / attendus` (taux de complétude), non stocké par série, sans prise en compte de la part reconstruite, de l'historique ou de la fiabilité de la source.
E6	Exploitable par les métiers	**Plutôt couvert**	Parcours en 5 pages, diagnostics en français, aperçu des corrections, historique, fenêtre des critères. Manques : exports, authentification, vue « territoire ».
E7	Réalités du terrain	**Mixte**	Trous : couvert. Multifluides : couvert. Compteurs renommés, bâtiments rénovés, valeurs estimées en source, fréquences saccadées : non gérés ou mal gérés (voir §4).

4. Défauts confirmés par exécution

4.1 Synthèse

#	Gravité	Constat	Emplacement
1	**Critique**	L'import plante dès qu'une cellule de valeur est vide.	`import_service.py:189-205`
2	**Critique**	La validation « lot » défait le traitement et remet les décisions à zéro.	`import_service.py:407`
3	Moyen	Incohérence de score entre l'API et l'écran après enregistrement d'un compteur.	`routers/imports.py:72` , `import_service.py:505`
4	**Haute**	Écriture de fichier hors du dossier de stockage (nom de fichier non assaini).	`import_service.py:116-118`
5	**Haute**	Faux positifs massifs de la règle « pic » en pas horaire.	`quality_engine.py:219-252`
6	Haute	La reconstruction par profil ignore la nature du compteur (index, livraison).	`reconstruction_engine.py:131-185`
7	Moyen	Fréquences atypiques ou mixtes non gérées.	`quality_engine.py:53-67`
8	Moyen	Rénovation (baisse structurelle) mal traitée.	`quality_engine.py:254-275`
9	Moyen	Valeurs déjà estimées en amont non détectées.	—
10	Moyen	Palier suspect : les premiers points ne sont pas signalés.	`quality_engine.py:297-306`
11	Moyen	Compteur renommé = deux compteurs ; analyse isolée par lot.	`import_service.py:56` , `:250`
12	Faible	Heure d'été : faux « relevé absent ».	`quality_engine.py:171`
13	Moyen	Performance d'import (insertions ligne à ligne).	`import_service.py:189-205`

4.2 Détail

1. Import impossible avec une valeur vide (critique). `iterrows()` sur un tableau mixte date/nombre convertit `NaN` en `NaT` ; SQLAlchemy lève `StatementError: float() argument must be ... not 'NaTType'`. La règle `valeur_nulle` décrite dans le README est donc inatteignable via l'import. Reproduit directement avec `parse_csv_bytes` (pandas 2.2.3, version épinglée).

2. La validation du lot annule le traitement (critique). Cas testé : 1 trou de 12 h et 24 zones « pic », toutes corrigées puis validées. - Avant validation : 40 points `reconstruite`. - Après `validate_and_reconstruct` : **0** `reconstruite` ; 2 864 valides, 28 incohérents, 12 manquants. - Les 25 anomalies repassent à `en_attente`, une anomalie `valeur_nulle` apparaît, le score reste 98,6 % → 98,6 %, le statut est « validé ».

Cause : `run_quality_analysis` est rappelée à la fin (`:407`) ; elle relit `raw_value` (`_meter_rows_df`, `:233`), supprime et recrée les anomalies (`:241`) et réécrit la qualité de tous les points (`:276-279`). Le bouton « Réanalyser » a le même effet sur les décisions déjà prises.

3. Scores incohérents (moyen). Après « enregistrer le compteur » : l'API annonce 100 %, l'écran affiche 99,5 % (les points `reconstruite` ne sont pas comptés comme valides). `points_smoothed` vaut 0 alors que 3 points ont été lissés, car `resolve_anomaly` les avait déjà modifiés.

4. Écriture hors dossier (haute). `os.path.join(batch_dir, filename)` avec `filename = "../../escaped.csv"` écrit le fichier hors du dossier du lot. Constaté. Aucune authentification n'est en place.

5. Faux positifs en pas horaire (haute). Benchmark : 2 ans de données saines avec cycle jour/nuit, 50 anomalies ponctuelles injectées.

Pas	Détectées	Points signalés à tort	Zones « pic »	Score
Horaire (17 520 pts)	50/50	**3 713 (21,2 %)**	848	**78,4 %**
Quart d'heure (70 080 pts)	50/50	315 (0,45 %)	20 (= injectés)	99,3 %

Cause : la fenêtre de référence est comptée en périodes (`reference_window_periods`, ±15) et non en durée ; en horaire, un point de midi est comparé à la phase opposée du cycle jour/nuit.

6. Reconstruction indifférente à la nature du compteur (haute). - Index : « profil type » propose 13 924 pour une valeur réelle de 13 038 (+6,8 %), ce qui créerait ensuite une « baisse d'index » ; l'interpolation donne 13 038,4. - Livraison manquante de 3 000 : reconstruite à **0,0** par la cascade par défaut. Seule l'interface propose des méthodes adaptées ; le moteur ne s'adapte pas au `measurement_kind`.

7. Fréquences (moyen). - Série 15 min puis horaire : granularité lue « 15 min », 2 085 « manquants » sur 5 757 attendus, score 63,8 %. - Pas de 10 min : lu comme du 15 min, 2 015 horodatages « irréguliers », **score 0,0 %**.

8. Rénovation (moyen). Baisse structurelle de 40 % à partir du 1er mars : 628 points sur 2 928 (21 %) signalés « consommation basse », répartis en 437 zones fragmentées. Ni rupture détectée, ni changement ignoré.

9. Valeurs estimées en amont (moyen). 48 h d'interpolation linéaire exacte injectées : aucune anomalie, 100 % « valides ».

10. Palier suspect (moyen). 10 valeurs identiques consécutives : 7 points signalés ; les 3 premiers restent « valides » (le signalement commence au quatrième).

11. Renommage et continuité (moyen). Renommer `C1-EL` en `C1-ELEC` crée deux compteurs distincts. Un même compteur importé en deux lots est unique en base, mais chaque lot est

analysé séparément (1 464 points au lieu de 2 904).

12. Heure d'été (faible). Le 02:00 inexistant du 31 mars est signalé « absence de relevé » (1 point par an). Le cas du retour d'heure (doublon écrasé par `keep="last"`) n'a pas été testé.

13. Performance (moyen). Ingestion + analyse de 35 040 points (1 compteur, 1 an en quart d'heure) : **15 s** ; l'analyse seule : 1,3 s. Extrapolation non mesurée : plus de 20 min pour 100 compteurs.

4.3 Points solides constatés

- Rappel de 100 % sur 50 anomalies ponctuelles injectées (20 pics, 20 chutes, 10 zéros) en horaire et en quart d'heure.
- 150 points manquants injectés sur 150 signalés (30 en horaire, 120 en quart d'heure).
- `raw_value` toujours conservé ; déduplication par empreinte ; classification multifluide ; seuils réglables.

4.4 Hygiène technique

- La CI ne lance pas `pytest` ; les tests ne couvrent pas la chaîne import → base → validation (d'où les défauts 1 à 3 non détectés).
- Reconstruction dupliquée entre `validate_and_reconstruct` et `validate_meter`.
- Alembic installé mais inutilisé : `main.py` applique des `ALTER TABLE` manuels.
- `Meter.code` sert de clé sans contrainte d'unicité.
- Pas de `package-lock.json` ; la CI utilise `npm install` (builds non reproductibles).
- API dépréciées : `Query.get`, `datetime.utcnow`, `@app.on_event`, `class Config` (pydantic).
- `get_import` associe le bâtiment du premier compteur seulement et interroge les relevés compteur par compteur.

5. Améliorations proposées

Tailles : S = quelques heures, M = quelques jours, L = plusieurs semaines. Estimations indicatives.

Phase 0 — Correctifs (priorité immédiate, S/M)

- Défaut 1 : convertir `NaN → None` à l'import (éviter `iterrows`).
- Défaut 2 : la réanalyse doit préserver les statuts `reconstruite` et les résolutions ; le score « après » doit se calculer sur `value`.
- Défaut 3 : compter `reconstruite` de façon cohérente entre API et écran.
- Défaut 4 : assainir les noms de fichiers (nom de base uniquement, ou identifiant généré).
- Ajouter un test de bout en bout import → validation et exécuter `pytest` dans la CI.
- Vérifier les données déjà traitées en base : leurs statuts peuvent être faux.

Phase 1 — Cœur du défi : statuts, traçabilité, confiance (M)

- **Statut par point** `mesuré / estimé / hypothèse` :
- mesuré : valeur source non signalée ;
- estimé : reconstruite par une méthode déterministe à partir des données du compteur ;
- hypothèse : valeur dépendant d'une supposition à confirmer (site fermé, remplacement de compteur, livraison non saisie).

- **Table `reading_adjustments`** : méthode, paramètres, version du moteur, anomalie d'origine, texte de l'hypothèse, auteur, date.
- **Score de confiance par série**, stocké et décomposé : pondération par statut (par exemple mesuré 1,0 ; estimé 0,5 ; hypothèse 0,2 — poids à calibrer), pénalités pour historique court, fréquence instable ou rupture non expliquée.

Phase 2 — Veille continue (L)

- Une série par compteur, tous imports confondus, analyse incrémentale sur fenêtre glissante.
- Endpoint d'ingestion ou dossier surveillé, planificateur, alertes.
- Vue « santé du territoire » agrégée par bâtiment et par fluide.
- Détection de ruptures communes à plusieurs compteurs partageant un même automate (pistes déjà citées dans le README).

Phase 3 — Réalités du terrain (M/L)

- Table de filiation des compteurs (ancien code → nouveau, dates) et table d'événements de bâtiment (rénovation, fermeture).
- Détection de rupture de profil (CUSUM ou PELT) pour ouvrir un nouveau régime de référence.
- Détecteur de valeurs estimées en source (colonne de statut, interpolation exacte, répétitions).
- Segmentation par régime de pas ; horodatages normalisés en UTC.
- Règle « pic » exprimée en durée (défaut 5) ; reconstruction guidée par `measurement_kind` (défaut 6).

Phase 4 — Rendre l'« IA » réelle (L)

- Le moteur actuel est un ensemble de règles ; `diagnosis_engine.py` le reconnaît. L'interface le présente comme un assistant « qui apprend », alors que rien n'apprend : à reformuler ou à rendre vrai.
- Pistes : modèle de référence par compteur (régression quantile heure × jour × météo), calibrage des seuils à partir des décisions « validée/corrigée », explications en langage naturel produites à partir des traces (le moteur déterministe restant la source de vérité).

Phase 5 — Exploitabilité (M)

- Export CSV/Excel/PDF du dossier de qualité avec traçabilité.
- Authentification, rôles, auteur dans l'historique.
- API de lecture exposant valeur + statut + confiance.

6. Recommandation

Commencer par la Phase 0, puis la Phase 1 : ce sont elles qui rendent le projet conforme à « qualifier chaque valeur et tracer chaque hypothèse ». La Phase 2 répond au mot « en continu ». Rejouer le benchmark sur les données kDrive réelles avant de figer les seuils.

Annexe — Reproduire les mesures

Environnement : Python 3.13, dépendances de `backend/requirements.txt` + `pytest`, pandas 2.2.3, base SQLite temporaire.

- Sondages : `/tmp/claude-0/-home-user-qualif-comptage/d0ad72fc-e1eb-5f80-ba17-d5a7a73e2b01/scratchpad/audit_probe.py`
- Benchmark de détection : `/tmp/claude-0/-home-user-qualif-comptage/d0ad72fc-e1eb-5f80-ba17-d5a7a73e2b01/scratchpad/bench.py`
- Exécution depuis `backend/` : `python -W ignore <script>`

Paramètres du benchmark : graine aléatoire 1 ; profil = cycle jour/nuit × semaine/week-end × saison, bruit 5 % ; `reference_window_periods=30` (valeur par défaut stockée dans `OrgSettings`).